Available online at www.sciencedirect.com
ScienceDirect
Information Processing Letters 107 (2008) 257–264
Information Processing Letters
www.elsevier.com/locate/ipl

Linear time algorithms for counting the number of minimal vertex covers with minimum/maximum size in an interval graph

Min-Sheng Lin *, Yung-Jui Chen

Department of Electrical Engineering, National Taipei University of Technology, 1, Sec. 3, Chung-Hsiao E. Rd., Taipei 106, Taiwan, ROC

Received 24 November 2007; received in revised form 4 February 2008; accepted 11 March 2008

Available online 18 March 2008

Communicated by M. Yamashita

Abstract

This paper presents efficient algorithms for an interval graph. These are (1) an algorithm for counting the number of minimum vertex covers, and (2) an algorithm for counting the number of maximum minimal vertex covers.

Keywords: Algorithms; Combinatorial problems; Counting problems; Interval graphs; Minimum vertex covers; Maximum minimal vertex covers

1. Introduction

Let $G = (V, E)$ be a graph, in which V and E denote vertex set and edge set of G, respectively. A subset $C \subseteq V$ is called a *vertex cover* of G if and only if every edge in E has at least one endpoint in C. The *size* of a vertex cover is the number of vertices in it. A vertex cover C is called *minimal* if and only if no proper subset of C is a vertex cover. A vertex cover with minimum size is called a *minimum vertex cover*. Note that a minimum vertex cover is always a minimal vertex cover but a minimal vertex cover may not be a minimum vertex cover. A minimal vertex cover with maximum size is called a *maximum minimal vertex cover*.

In graph theory, the notions of vertex cover and independent set are dual to each other. An independent set is a set of vertices in a graph no two of which are adjacent. Since every edge in E is incident on a vertex in a vertex cover C, no edge exists for which both endpoints are not in C. Thus, $V - C$ must be an independent set. Additionally, since minimizing C is the same as maximizing $V - C$, a minimum vertex cover defines a maximum independent set, and vice versa. This equivalence means that a program that solves the independent set problem can be used to solve the vertex cover problem.

However, counting the number of vertex cover (independent set) is a #P-complete problem [4]. Valiant [6] defined the class of #P-complete problems. The class of #P problems includes problems that involve counting access computations for problems in NP, while the class of #P-complete problems includes the hardest problems in #P. As is widely recognized, all known exact algorithms for solving these problems have exponential time complexity, making efficient algorithms for this class of problems unlikely to be developed. However, this complexity can be mitigated by considering only a restricted subclass of #P-complete problems. Okamoto, Uno and Uehara [3] proposed $O(|V| + |E|)$ time algorithms for counting the numbers of independent sets

* Corresponding author. Fax: +886 2 27317187.
E-mail address: mslin@ee.ntut.edu.tw (M.-S. Lin).

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2008.03.008

Table 1
Summary of the results

Counting problems	Chordal graphs		Interval graphs	
# of vertex covers	$O(n+m)$	[3]	$O(n)$	[2]
# of minimal vertex covers	#P-complete	[3]	$O(n)$	[2]
# of minimum vertex covers	$O(n+m)$	[3]	$O(n)$	[this paper]
# of maximum minimal vertex covers	#P-complete	[3]	$O(n)$	[this paper]

(vertex covers) and maximum independent sets (minimum vertex covers) in a chordal graph, and showed that the problem of counting the numbers of maximal independent sets (minimal vertex covers) and minimum maximal independent sets (maximum minimal vertex covers) remains #P-complete for a chordal graph. Our previous work [2] further considered only a subclass of chordal graphs–interval graphs, and yielded efficient algorithms in $O(|V|)$ time to count the numbers of vertex covers and minimal vertex covers in an interval graph. An *interval representation* of an undirected graph is a family of intervals that are assigned to the vertices such that vertices are adjacent if and only if the corresponding intervals intersect. An undirected graph G is called an *interval graph* if it has an interval representation. Numerous algorithms have been presented for interval graphs, and are used in various domains.

This paper presents fast and simple algorithms to count the numbers of minimum and maximum minimal vertex covers in an interval graph. Table 1 lists the comparisons of results of other papers [2,3] and this paper, where n is the number of vertices and m is the number of edges.

The next section introduces notation and preliminary results on which the rest of the paper is based. Section 3 presents an $O(n)$ time algorithm that counts the number of minimum vertex covers. Section 4 presents a straightforward $O(n^2)$ time algorithm for counting the number of maximum minimal vertex covers. Then, in Section 5, a double-ended queue for implementation is used, to develop a faster $O(n)$ time algorithm for counting the number of maximum minimal vertex covers.

2. Notation and preliminary results

Let $I = \{1, 2, \ldots, n\}$ denote the interval representation of the interval graph $G = (V, E)$. For simplicity, interval k in I corresponds to vertex k in V. Let a_k and b_k denote the left and right endpoints of interval k, respectively. Without loss of generality, assume that no two intervals share a common endpoint. The endpoints of all intervals are labeled with distinct integers between 1 and $2n$. The vertices from 1 to n are labeled according to their ascending right endpoints. That is, for two vertices i and j, $b_i < b_j$ if and only if $i < j$.

Let $I(p)$, for $1 \leqslant p \leqslant 2n$, be the interval whose endpoint (either left or right) is located at p. Therefore, if $I(p) = k$, then either $a_k = p$ or $b_k = p$. The existence of the function LR is assumed such that for any $p, 1 \leqslant p \leqslant 2n$, $LR(p) = \mathit{left}$ if p is a left endpoint; otherwise $LR(p) = \mathit{right}$. Some notation that will be used throughout this paper is introduced (see Table 2).

In our previous work [2], $x(k)$ and $y(k)$ are used to derive various linear-time algorithms. $x(k)$ represents the largest vertex to the left of vertex k that is not adjacent to vertex k, or equivalently, $x(k) = \max\{i \mid b(i) < a(k)\} = k - |N_k[k]|$. This equation implies that $N_k(k) = \{k-1, k-2, \ldots, x(k)+2, x(k)+1\}$. Given $I(p)$ and $LR(p)$, for $1 \leqslant p \leqslant 2n$, the following algorithm can be used to compute $x(k)$, for $1 \leqslant k \leqslant n$, in $O(n)$ time.

Algorithm *Compute_x_k*
  $x(0) \leftarrow 0$;
  $h \leftarrow 0$;
    // h is the rightmost interval for now //
  for $p \leftarrow 1$ **to** $2n$ **do**
    $k \leftarrow I(p)$;
    if $LR(p) = \mathit{left}$ **then** $x(k) \leftarrow h$;
      // p is the left endpoint of interval k //
      else $h \leftarrow k$;
      // p is the right endpoint of interval k //
  end-for
end-Algorithm

In our previous work [2], $y(k)$ is derived as $\max\{x(i) \mid i \in N_k[k]\}$; also, by the lemma proposed therein [2], $y(k) = \max(x(k), y(k-1))$, and the following algorithm can determine $y(k)$, for $1 \leqslant k \leqslant n$, in $O(n)$ time.

Algorithm *Compute_y_k*
  $y(0) \leftarrow -1$;
  for $k \leftarrow 1$ **to** n **do**
    if $y(k-1) > x(k)$ **then begin**
      $y(k) \leftarrow y(k-1)$;
    else
      $y(k) \leftarrow x(k)$;
    end-if-else
  end-for
end-Algorithm

Table 2

$G = (V, E)$	interval graph, where $\|V\| = n$ and $V = \{1, 2, \ldots, n\}$
G_k	subgraph induced by vertices $\{1, 2, \ldots, k\}$ with $k \leqslant n$
a_i, b_i	left and right endpoints of interval i (vertex i), respectively
p	endpoint between 1 and $2n$
$I(p)$	interval whose endpoint (either left or right) is located at p
$LR(p)$	$= \begin{cases} left & \text{if } p \text{ is a left endpoint} \\ right & \text{if } p \text{ is a right endpoint} \end{cases}$
$N_k(i)$	open neighborhood of vertex i in $G_k, i \leqslant k$
$N_k[i]$	closed neighborhood of vertex i in G_k. Namely, $N_k(i) \cup \{i\}$
$x(k)$	largest vertex on the left side of vertex k and not adjacent to vertex k, or equivalently, $x(k) = \max\{j \mid b_j < a_k\} = k - \|N_k[k]\| = k - \|N_k(k)\| - 1$
$y(k)$	$\max\{x(i) \mid i \in N_k[k]\}$
VC_k	set of all vertex covers in G_k
MVC_k	set of all minimal vertex covers in G_k
$MVC_k[\bar{i}]$	set of all minimal vertex covers in G_k excluding vertex i but including all vertices j with $i < j \leqslant k, \equiv \{C\colon C \in MVC_k$ and vertex $i \notin C$ and $\{$vertex $j \mid i < j \leqslant k\} \subset C\}$
$\alpha(k)$	minimum size of vertex covers in $G_k, \equiv \min\{\|C\|\colon C \in VC_k\}$
$\beta(k)$	maximum size of minimal vertex covers in $G_k, \equiv \max\{\|C\|\colon C \in MVC_k\}$
$\beta(k, i)$	maximum size of minimal vertex covers in G_k excluding vertex i but including all vertices j with $i < j \leqslant k, \equiv \max\{\|C\|\colon C \in MVC_k[\bar{i}]\}$
$\#\alpha(k)$	number of minimum vertex covers in $G_k, \equiv \|\{C\colon C \in VC_k$ and $\|C\| = \alpha(k)\}\|$
$\#\beta(k)$	number of maximum minimal vertex covers in $G_k, \equiv \|\{C\colon C \in MVC_k$ and $\|C\| = \beta(k)\}\|$
$\#\beta(k, i)$	number of the maximum minimal vertex covers in G_k excluding vertex i but including all vertices j with $i < j \leqslant k, \equiv \|\{C\colon C \in MVC_k[\bar{i}]$ and $\|C\| = \beta(k, i)\}\|$

For more details on $x(k)$ and $y(k)$, please refer to our previous paper [2].

3. $O(n)$-time algorithm to count the number of minimum vertex covers

This section presents an $O(n)$ time algorithm for counting the number of minimum vertex covers in an interval graph G. A recurrence equation for $\alpha(k)$, the minimum size of the vertex covers in G_k, is presented first.

Lemma 3.1. *For* $1 \leqslant k \leqslant n$, $\alpha(k) = \min(\alpha(k-1) + 1, \alpha(x(k)) + k - x(k) - 1)$.

Proof. Let C represent a vertex cover of G_k. If vertex $k \in C$, then clearly, C is a vertex cover of G_k with vertex $k \in C$ if and only if $C - \{k\}$ is a vertex cover of G_{k-1}. Therefore, for any vertex cover C' of G_{k-1}, $C' \cup \{k\}$ is a vertex cover of G_k. If vertex $k \notin C$, then according to the definition of the vertex cover, $N_k(k)$ is a subset of C and $C - N_k(k)$ is a vertex cover of $G_{x(k)}$. Conversely, for any vertex cover C'' of $G_{x(k)}$, $C'' \cup N_k(k)$ is a vertex cover of G_k. Hence,

$$\begin{aligned}
\alpha(k) &= \min\{|C|\colon C \in VC_k\} \\
&= \min\Big(\{|C' \cup \{k\}|\colon C' \in VC_{k-1}\} \\
&\qquad \cup \{|C'' \cup N_k(k)|\colon C'' \in VC_{x(k)}\}\Big) \\
&= \min\Big(\min\{|C' \cup \{k\}|\colon C' \in VC_{k-1}\}, \\
&\qquad \min\{|C'' \cup N_k(k)|\colon C'' \in VC_{x(k)}\}\Big) \\
&= \min\Big(\min\{|C'|\colon C' \in VC_{k-1}\} + |\{k\}|, \\
&\qquad \min\{|C''|\colon C'' \in VC_{x(k)}\} + |N_k(k)|\Big) \\
&= \min\big(\alpha(k-1) + 1, \alpha(x(k)) + k - x(k) - 1\big),
\end{aligned}$$

where the last equality holds since $x(k) = k - |N_k(k)| - 1$. □

Based on Lemma 3.1, the following algorithm can be applied to compute $\alpha(k)$, for $1 \leqslant k \leqslant n$, in $O(n)$ time.

Algorithm *Compute_*α_k
  $\alpha(0) \leftarrow 0$;
  for $k \leftarrow 1$ **to** n **do**
    if $\alpha(k-1) + 1 > \alpha(x(k)) + k - x(k) - 1$ **then begin**
      $\alpha(k) \leftarrow \alpha(x(k)) + k - x(k) - 1$;
    else
      $\alpha(k) \leftarrow \alpha(k-1) + 1$;
    end-if-else
  end-for
end-Algorithm

Now, the number of vertex covers with the minimum size in G_k, for $1 \leqslant k \leqslant n$, can be obtained, and the time-complexity is also $O(n)$.

Lemma 3.2. *For* $1 \leqslant k \leqslant n$,

$$\#\alpha(k) = \begin{cases} \#\alpha(k-1) \\ \quad \text{if } \alpha(k-1)+1 < \alpha(x(k))+k-x(k)-1, \\ \#\alpha(x(k)) \\ \quad \text{if } \alpha(k-1)+1 > \alpha(x(k))+k-x(k)-1, \\ \#\alpha(k-1)+\#\alpha(x(k)) \\ \quad \text{if } \alpha(k-1)+1 = \alpha(x(k))+k-x(k)-1. \end{cases}$$

Proof. Firstly, $\alpha(k-1)+1 < \alpha(x(k))+k-x(k)-1$ is considered. Let the set of all minimum vertex covers in G_{k-1} be $M' = \{C' \colon C' \in VC_{k-1} \text{ and } |C'| = \alpha(k-1)\}$. From the proof of Lemma 3.1, a minimum vertex cover C in G_k consists of vertex k and a minimum vertex cover C' in G_{k-1}. Restated, the set of all minimum vertex covers in G_k is $M = \{C' \cup \{k\} \colon C' \in M'\}$. Therefore, the number of minimum vertex covers in G_k is the same as that in G_{k-1} and $\#\alpha(k) = \#\alpha(k-1)$. Next, $\alpha(k-1)+1 > \alpha(x(k))+k-x(k)-1$ is considered. Similarly, from the proof of Lemma 3.1, a minimum vertex cover C in G_k consists of the neighborhood of vertex k and a minimum vertex cover C'' in $G_{x(k)}$. Hence, the number of minimum vertex covers in G_k is the same as that in $G_{x(k)}$ and $\#\alpha(k) = \#\alpha(x(k))$. Finally, $\alpha(k-1)+1 = \alpha(x(k))+k-x(k)-1$ is considered. At this time, the set of all minimum vertex covers in G_k are obtained from the foregoing cases. Therefore, $\#\alpha(k) = \#\alpha(k-1)+\#\alpha(x(k))$. □

The $O(n)$ time algorithm is described in detail as follows.

Algorithm *Count_#α_k*
  #α(0) ← 1;
  for k ← 1 **to** n **do**
    if α(k−1)+1 < α(x(k))+k−x(k)−1 **then begin**
      #α(k) ← #α(k−1);
    else if α(k−1)+1 > α(x(k))+k−x(k)−1 **then begin**
      #α(k) ← #α(x(k));
    else
      #α(k) ← #α(k−1)+#α(x(k));
    end-if-else
  end-for
  output("The number of minimum vertex covers is", #α(n));
end-Algorithm

Theorem 3.1. *The number of minimum vertex covers in an interval graph can be counted in* $O(n)$ *time.*

4. $O(n^2)$-time algorithm to count the number of maximum minimal vertex covers

This section provides a simple algorithm, but in the worst-case $O(n^2)$ time, for counting the number of maximum minimal vertex covers. Lin [2] established the following equation,

$$MVC_k = MVC_k[\bar{k}] \cup MVC_k[\overline{k-1}] \cup \cdots \cup MVC_k\big[\overline{y(k)+1}\big]. \tag{4.1}$$

By the definition of the maximum minimal vertex cover and Eq. (4.1),

$$\beta(k) = \max_{i=y(k)+1}^{k} \beta(k,i). \tag{4.2}$$

According to the definition of the vertex cover, if a vertex cover does not contain the vertex i, it must contain the neighborhood of the vertex i. Therefore,

$$MVC_k[\bar{i}] = \big\{\{k, k-1, \ldots, i+1\} \cup N_k(i) \cup C' \colon C' \in MVC_{x(i)}\big\}. \tag{4.3}$$

Hence, $\beta(k,i)$ in Eq. (4.2) can be obtained in constant time by the following lemma.

Lemma 4.1. *For* $1 \leqslant k \leqslant n$,

$$\beta(k,i) = \begin{cases} \beta(x(i))+k-x(i)-1 & \text{if } y(k)+1 \leqslant i \leqslant k, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. By the definition of $\beta(k,i)$ and Eq. (4.3), we have

$$\begin{aligned} \beta(k,i) &= \max\big\{|C| \colon C \in MVC_k[\bar{i}]\big\} \\ &= \max\big\{|\{k, k-1, \ldots, i+1\} \cup N_k(i) \cup C'| \colon \\ &\qquad C' \in MVC_{x(i)}\big\}. \end{aligned}$$

Since $\{k, k-1, \ldots, i+1\} \cup N_k(i) = \{k, k-1, \ldots, i+1\} \cup N_i(i)$ and $\{k, k-1, \ldots, i+1\} \cap N_i(i) = \emptyset$, we have

$$\begin{aligned} \beta(k,i) &= \big|\{k, k-1, \ldots, i+1\}\big| + \big|N_i(i)\big| \\ &\qquad + \max\big\{|C'| \colon C' \in MVC_{x(i)}\big\} \\ &= (k-i) + \big(i - x(i) - 1\big) + \beta\big(x(i)\big) \\ &= k - x(i) - 1 + \beta\big(x(i)\big). \end{aligned}$$

□

According to Eq. (4.2) and Lemma 4.1, the following straightforward algorithm can be adopted to compute $\beta(k)$, for $1 \leqslant k \leqslant n$, in $O(n^2)$ time.

Algorithm *Compute_β_k*
  β(0) ← 0;
  for k ← 1 **to** n **do**
    β(k) ← −1;
    for i ← y(k)+1 **to** k **do**
      if β(k,i) > β(k) **then**
        β(k) ← β(k,i);
          // by Lemma 4.1, the value of β(k,i)
            can be obtained in constant time //

```
    end-for
  end-for
end_Algorithm
```

Lemma 4.2. *For* $1 \leqslant k \leqslant n$ *and* $y(k)+1 \leqslant i \leqslant k$, $\#\beta(k,i) = \#\beta(x(i))$.

Proof. From the proof of Lemma 4.1, a one-to-one correspondence clearly exists between $MVC_k[\bar{i}]$ and $MVC_{x(i)}$. Therefore, the number of all minimal vertex covers with the maximum size $\beta(k,i)$ in $MVC_k[\bar{i}]$ equals that with the maximum size $\beta(x(i))$ in $MVC_{x(i)}$ and $\#\beta(k,i) = \#\beta(x(i))$. Hence, the lemma holds. □

Lemma 4.3. *For* $1 \leqslant k \leqslant n$,

$$\#\beta(k) = \sum_{i=y(k)+1}^{k} \#\beta\big(x(i)\big) \cdot t(k,i), \quad \textit{where}$$

$$t(k,i) = \begin{cases} 1 & \textit{if } \beta(k,i) = \beta(k), \\ 0 & \textit{otherwise}. \end{cases}$$

Proof. According to Eq. (4.1) and Lemma 4.2, $\#\beta(k)$ can be accumulated through $\#\beta(x(i))$ by checking whether each $\beta(k,i)$ equals $\beta(k)$ for $y(k)+1 \leqslant i \leqslant k$. □

The counting $\#\beta(k)$ algorithm in worst-case $O(n^2)$ time is described in detail as follows.

```
Algorithm Count_#β_k
  #β(0) ← 1;
  for k ← 1 to n do
    #β(k) ← 0;
    for i ← y(k) + 1 to k do
      if β(k, i) = β(k) then
        #β(k) ← #β(k) + #β(x(i));
    end-for
  end-for
  output("The number of maximum minimal vertex covers
          is", #β(n));
end-Algorithm
```

5. $O(n)$-time algorithm to count the number of maximum minimal vertex covers

Section 4 presents the straightforward dynamic programming approach for Eq. (4.2), which takes time $O(n^2)$. This section first presents some properties to accelerate dynamic programming and then proposes an $O(n)$ time algorithm.

Property 5.1. *If* $k < k'$, *then*

$$y(k) \leqslant y(k').$$

Proof. According to Lemma 3.2 in [2], $y(k) = \max(x(k), y(k-1))$, for $1 \leqslant k \leqslant n$. Thus, this property holds. □

Property 5.2. *For each* i *with* $y(k)+1 \leqslant i \leqslant k-1$,

$$\beta(k,i) = \beta(k-1,i)+1 \quad \textit{and}$$
$$\#\beta(k,i) = \#\beta(k-1,i).$$

Proof. According to Property 5.1, $y(k-1) \leqslant y(k)$. Hence, $y(k-1)+1 \leqslant y(k)+1 \leqslant i \leqslant k-1$. Then, by Lemma 4.1, $\beta(k-1,i) = \beta(x(i)) + k - x(i) - 2$. Clearly, $\beta(k,i) = \beta(x(i)) + k - x(i) - 1$ for $y(k)+1 \leqslant i \leqslant k-1 < k$. Thus, $\beta(k,i) = \beta(k-1,i)+1$. By Lemma 4.2, $\#\beta(k,i) = \#\beta(k-1,i) = \#\beta(x(i))$, for $y(k)+1 \leqslant i \leqslant k-1$. Therefore, this property follows.

Property 5.3. *For* $1 \leqslant k \leqslant n$,

$$\beta(k) = \max\Big(\max_{i=y(k)+1}^{k-1} \big\{\beta(k-1,i)+1\big\}, \beta(k,k)\Big).$$

Proof. By Eq. (4.2), the computation of

$$\beta(k) = \max_{i=y(k)+1}^{k} \beta(k,i) \quad \text{and}$$

$$\beta(k-1) = \max_{i=y(k-1)+1}^{k-1} \beta(k-1,i)$$

is treated as a competition among candidate vertices $y(k)+1, y(k)+2, \ldots, k$ and candidate vertices $y(k-1)+1, y(k-1)+2, \ldots, k-1$, respectively. Based on Property 5.1, $y(k-1)+1 \leqslant y(k)+1$. Accordingly, the overlapping candidate vertices for computing both $\beta(k)$ and $\beta(k-1)$ are those vertices i such that $y(k)+1 \leqslant i \leqslant k-1$, and $\beta(k)$ has one more candidate, vertex k, to be considered. Therefore, $\beta(k) = \max\{\max_{i=y(k)+1}^{k-1}\{\beta(k,i)\}, \beta(k,k)\}$. By Property 5.2, $\beta(k,i) = \beta(k-1,i)+1$, for $y(k)+1 \leqslant i \leqslant k-1$. The proof is completed. □

Property 5.3 implies that at iteration k, $\beta(k)$ can be computed simply by comparing only the value of $\beta(k,k)$ from the new candidate vertex k with the value of $\beta(k-1,i)+1$ ($\equiv \beta(k,i)$) from the old candidate vertex i, for $y(k)+1 \leqslant i \leqslant k-1$, generated at iteration $k-1$. Furthermore, if the newly added candidate vertex k is better than the old candidate vertex i, meaning $\beta(k,k) > \beta(k,i)$, then candidate vertex k remains better than candidate vertex i at a subsequent iteration $k' > k$. Hence, vertex i can be discarded and no longer treated as a candidate. This property can be stated as follows.

Property 5.4. *For* $y(k)+1 \leqslant i \leqslant k-1$, *if candidate vertex* k *is better than candidate vertex* i, *meaning*

empty(Q)	::= create an empty deque Q; F is set to one and R is set to zero
is_not_empty(Q)	::= if $F \leqslant R$, then return *true*; else return *false*
push_front(Q, v)	::= insert vertex v at the front of Q and set F to $F-1$
pop_front(Q)	::= remove and return the first vertex of Q and set F to $F+1$
push_rear(Q, v)	::= insert vertex v at the rear of Q and set R to $R+1$
pop_rear(Q)	::= remove and return the last vertex of Q and set R to $R-1$
$Q[q]$	::= return the vertex associated with index q in Q

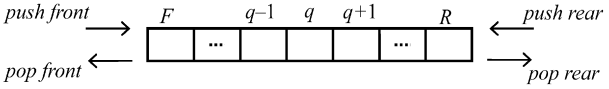

Fig. 1. Double-ended queue (deque) Q.

$\beta(k,k) > \beta(k,i)$, then candidate vertex k remains better than candidate vertex i at a subsequent iteration $k' > k$.

Proof. By Lemma 4.1, $\beta(k,i) = \beta(x(i)) + k - x(i) - 1$ and $\beta(k,k) = \beta(x(k)) + k - x(k) - 1$. Since $\beta(k,k) > \beta(k,i)$, we have $x(i) - x(k) + \beta(x(k)) - \beta(x(i)) > 0$. Two cases must be considered.

Case 1. $y(k') + 1 \leqslant i < k'$. By Lemma 4.1, $\beta(k',k) - \beta(k',i) = x(i) - x(k) + \beta(x(k)) - \beta(x(i)) > 0$. Thus, $\beta(k',k) > \beta(k',i)$.

Case 2. $i < y(k') + 1$. Vertex i is not a valid candidate for computing $\beta(k')$ at iteration k'.

The proof is completed. □

Rather than simply searching for the best one among all candidates for computing $\beta(k)$ at iteration k, an efficient data structure, called the double-ended queue (deque), is used to maintain candidate vertices that were derived from the above properties. A deque can have items added to or removed from either end, as shown in Fig. 1. These operations applied to a deque Q are specified as follows. In growing array implementation, the time complexity of all operations is $O(1)$. Let F and R be the indices of the front and rear items in Q, respectively.

The content of Q is obtained iteratively. At the kth iteration, according to Property 5.4, before the new candidate vertex k is added to the rear of Q, the removal of old candidate vertex $Q[R]$ from the rear of Q is repeated until $\beta(k, Q[R]) \geqslant \beta(k,k)$. Thus, at the end of each iteration k, the content of Q should satisfy the following properties.

Property 5.5. *$Q[F] \leqslant Q[q_1] < Q[q_2] \leqslant Q[R]$, for all q_1, q_2 and $F \leqslant q_1 < q_2 \leqslant R$.*

Property 5.6. *$\beta(k, Q[F]) \geqslant \beta(k, Q[q_1]) \geqslant \beta(k, Q[q_2]) \geqslant \beta(k, Q[R])$, for all q_1, q_2 and $F \leqslant q_1 < q_2 \leqslant R$.*

Properties 5.5 and 5.6 imply that vertices in Q are increasing and their $\beta(k,i)$ values are non-increasing. Therefore, at the end of each iteration k, the leftmost candidate vertex j in Q for which $j \geqslant y(k) + 1$ will have the maximum value of $\beta(k,j)$ among all candidates for computing $\beta(k)$. In fact, by Property 5.1, $y(k) + 1 \leqslant y(k') + 1$ for $k < k'$, so the candidate vertex i such that $i < y(k) + 1$ (implying $i < y(k') + 1$) can be removed from the front of Q and no longer treated as a candidate, as has been shown in case 2 in the proof of Property 5.4.

Based on the above discussion, the main steps in each iteration k for the new algorithm to count the number of maximum minimal vertex covers in G_k are described in detail as follows, and then are verified to work correctly.

Step 1. Based on Property 5.6, after the end of iteration $k-1$, since $\beta(k, Q[F])$ is the maximum value of all $\beta(k,i), i \in Q$, whether $\beta(k,k)$ exceeds $\beta(k, Q[F])$ is checked first. If $\beta(k,k)$ exceeds $\beta(k, Q[F])$, then $\beta(k,k)$ exceeds $\beta(k,i)$ for all $i \in Q$. Then, Q is cleaned. Secondly, add vertex k to Q. Finally, assign $\beta(k,k)$ and $\#\beta(k,k)$ to $\beta(k)$ and $\#\beta(k)$, respectively. Terminate this iteration and proceed to the next iteration. Otherwise, perform the following steps.

Step 2. Firstly, $\#\beta(k)$ is initialized to $\#\beta(k-1)$. Next, check whether Q is empty and whether $Q[F]$ is less than $y(k) + 1$. If Q is not empty and $Q[F]$ is less than $y(k) + 1$, i.e. $Q[F]$ is not a candidate for computing $\beta(k)$, then next check whether $\beta(k-1, Q[F])$ equals $\beta(k-1)$. If $\beta(k-1, Q[F])$ equals $\beta(k-1)$, then subtract $\#\beta(k-1, Q[F])$ from $\#\beta(k)$ and remove $Q[F]$ from Q. Repeat the first check in Step 2 until $Q[F]$ is not less than $y(k) + 1$ or Q is empty.

Step 3. By Property 5.4, before inserting vertex k into the rear of Q, check whether $\beta(k, Q[R])$ is less than

$\beta(k,k)$. If $\beta(k, Q[R])$ is less than $\beta(k,k)$, then remove $Q[R]$ from Q. Repeat Step 3 until $\beta(k, Q[R])$ is not less than $\beta(k,k)$ or Q is empty. Finally, insert vertex k into the rear of Q as the final item in Q.

Step 4. After finishing Step 3, $Q[F]$ is the best candidate vertex in Q and set $\beta(k)$ be $\beta(k, Q[F])$.

Step 5. Finally, calculate $\#\beta(k)$. Two cases are distinguished.

Case 1. $\#\beta(k)$ equals zero.

Recalculate $\#\beta(k)$ by means of Q. Scan the vertex i in Q from front to rear, and accumulate $\#\beta(k,i)$ for the vertex i with $\beta(k,i)$ equal to $\beta(k)$. Thus, the final accumulated result is $\#\beta(k)$.

Case 2. $\#\beta(k)$ does not equal zero and $\beta(k,k)$ equals $\beta(k)$.

$$\#\beta(k) = \#\beta(k) + \#\beta(k,k).$$

The above steps maintain Q to satisfy the following properties.

Property 5.7. *For each iteration k, $1 \leqslant k \leqslant n$, after Step 2 has been executed*;

$$Q[F-1] < y(k)+1 \leqslant Q[F],$$

where $Q[F-1]$ is the largest item in Q which is less than $y(k)+1$.

Property 5.8. *For each iteration k, $1 \leqslant k \leqslant n$, after Step 3 has been executed*;

$$\max_{i=Q[q]+1}^{Q[q+1]} \beta(k,i) = \beta\big(k, Q[q+1]\big),$$

for $F-1 \leqslant q < R$.

When $q = F-1$, by Property 5.8,

$$\max_{i=Q[F-1]+1}^{Q[F]} \beta(k,i) = \beta\big(k, Q[F]\big),$$

and by Property 5.7,

$$Q[F-1] < y(k)+1 \leqslant Q[F].$$

Thus, the following property is exhibited.

Property 5.9. *For each iteration k, $1 \leqslant k \leqslant n$, after Step 3 has been executed*;

$$\max_{i=y(k)+1}^{Q[F]} \beta(k,i) = \beta\big(k, Q[F]\big).$$

Lemma 5.1. *For each iteration k, $1 \leqslant k \leqslant n$, after Step 3 has been executed*;

$$\beta(k) = \beta\big(k, Q[F]\big).$$

Proof. By Eq. (4.2), $\beta(k) = \max_{i=y(k)+1}^{k} \beta(k,i)$. At the end of Step 3, since vertex k was inserted into the rear of Q as the final item in Q, $Q[R] = k$. Moreover, by Property 5.7, $y(k)+1 \leqslant Q[F]$. Hence, $\beta(k)$ can be rewritten as

$$\beta(k) = \max\Big(\max_{i=y(k)+1}^{Q[F]} \beta(k,i), \max_{i=Q[F]+1}^{Q[F+1]} \beta(k,i), \max_{i=Q[F+1]+1}^{Q[F+2]} \beta(k,i), \ldots, \max_{i=Q[R-1]+1}^{Q[R]=k} \beta(k,i) \Big).$$

By Properties 5.8 and 5.9,

$$\beta(k) = \max\big(\beta\big(k, Q[F]\big), \beta\big(k, Q[F+1]\big), \beta\big(k, Q[F+2]\big), \ldots, \beta\big(k, Q[R]\big)\big).$$

By Property 5.6, $\beta(k) = \beta(k, Q[F])$. □

Now, consider computing $\#\beta(k)$. By Property 5.2, if vertex i is a candidate for both iteration k and iteration $k-1$, i.e. $y(k)+1 \leqslant i \leqslant k-1$, then $\#\beta(k,i) = \#\beta(k-1,i)$. So, initially $\#\beta(k)$ is set to $\#\beta(k-1)$ in Step 2. The indices of the vertices that are less than $y(k)+1$ do not need to be listed to count the number of maximum minimal vertex covers. Therefore, those vertices i for $i < y(k)+1$ were deleted from Q, and if for vertex i, $\beta(k-1,i) = \beta(k-1)$, then $\#\beta(k-1,i)$ is subtracted from $\#\beta(k)$ in Step 2.

If all the vertices i with $\beta(k-1,i) = \beta(k-1)$ are removed from Q in Step 2, then $\#\beta(k)$ inherits no information from $\#\beta(k-1)$, i.e. $\#\beta(k) = 0$. In fact, if $\#\beta(k) = 0$ after the execution of Step 2, then the vertices i for which $\beta(k-1,i) = \beta(k-1)$ and the vertices j for which $\beta(k,j) = \beta(k)$ are non-overlapping. Hence, in this case, the new $\#\beta(k)$ must be recalculated from Q, as Case 1 in Step 5.

Otherwise, if $\#\beta(k) \neq 0$, then $\#\beta(k)$ can inherit from $\#\beta(k-1)$. Additionally, the effect of the new added vertex k must be taken into account in the calculation of $\#\beta(k)$, as Case 2 in Step 5.

The algorithm described above is now more formally described in Algorithm *Fast_Count_*$\#\beta_k$.

Notably, by Lemmas 4.1 and 4.2, the values of $\beta(k,i)$ and $\#\beta(k,i)$ in the above algorithm can be obtained as $\beta(x(i)) + k - x(i) - 1$ and $\#\beta(x(i))$, respectively, in constant time.

Algorithm *Fast_Count_*$\#\beta_k$

```
 1. β(0) ← 0;   // initial condition //
 2. #β(0) ← 1;   // initial condition //
 3. empty(Q);   //create an empty deque Q//
 4. push_rear(Q, 0);   // initial condition //
 5. for k ← 1 to n do
 6.     if β(k, k) > β(k, Q[F]) then begin   // Step 1 //
 7.         empty(Q);
 8.         push_rear(Q, k);
 9.         β(k) ← β(k, k);
10.         #β(k) ← #β(k, k);
11.     else begin   // Step 2 //
12.         #β(k) ← #β(k − 1);
13.         while is_not_empty(Q) and Q[F] < y(k) + 1 do
14.             if β(k − 1, Q[F]) = β(k − 1) then
                #β(k) ← #β(k) − #β(k − 1, Q[F]);
15.             pop_front(Q);
16.         end-while
17.         while is_not_empty(Q) and β(k, Q[R]) < β(k, k) do
                // Step 3 //
18.             pop_rear(Q);
19.         end-while
20.         push_rear(Q, k);
21.         β(k) ← β(k, Q[F]);   // Step 4 //
22.         if #β(k) = 0 then begin   // Step 5: Case 1 //
23.             q ← 0;
24.             while F + q ⩽ R and β(k, Q[F + q]) = β(k) do
25.                 #β(k) ← #β(k) + #β(k, Q[F + q]);
26.                 q ← q + 1;
27.             end-while
28.         else if β(k) = β(k, k) then begin   // Step 5: Case 2 //
29.             #β(k) ← #β(k) + #β(k, k);
30.         end-if-else
31.     end-if-else
32. end-for
33. output("The number of maximum minimal vertex covers is",
        #β(n));
end-Algorithm
```

Theorem 5.1. *The number of maximum minimal vertex covers in an interval graph can be counted in* $O(n)$ *time.*

Proof. Each vertex is pushed into Q exactly once (lines 8 or 20). In each iteration of the first two **while** loops (lines 13–16 and 17–19), exactly one *pop* operation (lines 15 and 18) is performed for each vertex that was previously pushed onto Q. Since the **for** loop is iterated only n times, the *push* operation is also executed n times in total. Hence, the *pop* operation cannot be executed more than n times in total. Consequently, the first two **while** loops are iterated at most n times over the whole algorithm. Consider executing the third **while** loop (lines 24–27) during the $(k-1)$th iteration of the **for** loop. Assume that the body of the third **while** loop is executed q times and let $X = \{Q[F], Q[F+1], \ldots, Q[F+q-1]\}$ be the set of the first q vertices in Q. This result implies that each vertex v in X has $\beta(k-1, v) = \beta(k-1)$ (line 24) and $\#\beta(k-1)$ is accumulated by those $\#\beta(k-1, v)$ (line 25). Next, consider the next iteration of the **for** loop, which is the kth iteration. $\#\beta(k)$ is either reset to $\#\beta(k, k)$ (line 10) or it is reset to the terminal value of the previous iteration (when the statement $\#\beta(k) \leftarrow \#\beta(k-1)$ is executed at line 12). In the former case, remove all vertices in Q (line 7). In the latter case, execute the first **while** loop to remove the first vertex, v, from Q until $v \geqslant y(k) + 1$ (line 15). During the execution of the first **while** loop, if $\beta(k-1, v) = \beta(k-1)$, such that vertex $v \in X$, then subtract $\#\beta(k-1, v)$ from $\#\beta(k)$. The condition $\#\beta(k) = 0$ is checked (line 22) to begin executing the third **while** loop. If the condition $\#\beta(k) = 0$ is true, then all vertices in X obtained from the $(k-1)$th iteration must have been removed from Q. Accordingly, the body of the third **while** loop is executed at most once for each vertex. Therefore, the third **while** loop is iterated at most n times over the whole algorithm, and the computational complexity of the above algorithm is $O(n)$. □

Ramalingam et al. [5] previously gave an $O(m+n)$ algorithm for computing a minimum-weighted independent dominating set for an interval graph. Later, Chang [1] derived an $O(n)$ time method when a sorted list of interval endpoints is given. The approach proposed herein is similar to that described in [1]. However, this approach differs in three aspects. First, the recursive formula, Eq. (4.2), is much simpler. Second, the approach follows own earlier work [2] in which $x(k)$ and $y(k)$ are used to derive various linear-time algorithms. And third, the maximum size, $\beta(k)$, is computed and its number, $\#\beta(k)$, is counted simultaneously herein.

References

[1] M.S. Chang, Efficient algorithm for the domination problems on interval and circular-arc graphs, SIAM J. Comput. 27 (1998) 1671–1694.

[2] M.S. Lin, Fast and simple algorithms to count the number of vertex covers in an interval graph, Inform. Process. Lett. 102 (2007) 143–146.

[3] Y. Okamoto, T. Uno, R. Uehara, Linear-time counting algorithms for independent sets in chordal graphs, in: Proceedings of 31st International Workshop on Graph-Theoretic Concepts in Computer Science, 2005, pp. 433–444.

[4] J.S. Provan, M.O. Ball, The complexity of counting cuts and of computing the probability that a graph is connected, SIAM J. Comput. 12 (4) (1983) 777–788.

[5] G. Ramalingam, C. Pandu Rangan, A unified approach to domination problems on interval graphs, Inform. Process. Lett. 27 (1988) 271–274.

[6] L.G. Valiant, The complexity of enumeration and reliability problems, SIAM J. Comput. 8 (1979) 410–421.